Slide 1 (intro):

Good morning everyone. Today I will present the current progress of my research project, which investigates decoding strategies for planning through autoregressive sequence modelling, under the supervision of Dr. Shane Josias.

Slide 2 (high level explanation of planning):

At a high level, planning means finding an ordered or partially ordered sequence of actions that changes an environment from an initial state to a desired goal state. For intuition, imagine a coffee machine. The initial state might be an empty cup in the machine, and the goal state might be a ready-to-drink cup of coffee. Planning then means deciding which actions to take, and in what order, such as grinding beans, adding water, and starting the brew cycle, so that the environment changes from the initial state to the goal state.

Slide 3 (why learn planning from data):

Why would we want a machine to solve planning problems automatically? Because many real systems must repeatedly decide what to do next in order to reach a goal. In simple settings, a person can write down the rules directly. But in more complicated settings, it can be useful to learn patterns for good decisions from examples rather than specifying every case by hand.

In this project, the examples are successful maze trajectories. A model is shown many valid ways of moving from a start position to a goal position. The hope is that, after seeing enough examples, it will learn regularities in those trajectories and use them when faced with a new query. In other words, it tries to capture what successful paths tend to look like.

So rather than hand-coding a separate rule for every possible start and goal pair, the aim is to let the model learn from demonstrations of successful planning behaviour.

Slide 5 (This work's environment):

Because real planning systems can be very complicated, this project studies a simpler setting. The environment is a discrete 2D maze represented as a grid. Some squares are free and some are walls. A move can go only up, down, left, or right. Given a start square and a goal square, the planning problem is to generate a valid sequence of moves that reaches the goal without passing through walls. In this talk I will refer to such a sequence as a trajectory.

Slide 6 (Imitation learning in the maze):

The model is trained from a dataset of successful trajectories rather than by trial and error inside the maze. Each trajectory is a sequence of maze positions together with the moves that connect them. So the model learns by imitating examples of good behaviour

that are already known to solve the planning task. This keeps the training problem closely tied to the valid solutions already available in the dataset.

Slide 7 (Generative modelling):

The model class used here is a generative model. Broadly, this means a model that learns a probability distribution from data. In this project, the data are valid maze trajectories. So after training, the model assigns probabilities to possible next moves, where a higher probability means that the move is judged to be more consistent with the successful trajectories in the training data. Those probabilities are then used by the decoding algorithm to decide how to continue the path.

Slide 8 (Autoregressive sequence modelling):

A direct probability distribution over all complete trajectories would be very difficult to work with, because the number of possible trajectories grows very quickly. Instead, the problem is simplified by predicting one move at a time. At each step, the model looks at the maze, the start, the goal, and the trajectory prefix generated so far, and then produces probabilities for the possible next moves. After one move is chosen, the position updates, and the process repeats until the goal is reached or a step limit is exceeded. That is what autoregressive sequence modelling means in this project: the path is built one step at a time, always using what has already been generated.

Slide 9 decoding algorithms:

The main research question then becomes: once the model gives probabilities for the next move, how should we convert those probabilities into an actual trajectory? That decision process is called decoding. In this project I compare three decoding strategies: greedy decoding, beam search with width 2, and beam search with width 3. The model provides the probabilities, but the decoding rule determines how those probabilities are turned into a concrete sequence of moves.

Slide 10 greedy decoding:

Greedy decoding is the simplest strategy. At each step, it chooses the single most likely next move. So after the first decision, it keeps one partial trajectory of length one. After the second decision, it has one partial trajectory of length two. After the third decision, it has one partial trajectory of length three, and so on. In other words, greedy decoding follows only one candidate path throughout the entire rollout. This is efficient and deterministic, but it is also short-sighted. A move that looks best right now is not always part of the best overall trajectory, so greedy decoding can miss stronger complete solutions.

Slide 11 beam search:

Beam search addresses that limitation by keeping several partial trajectories at each step instead of only one. If the beam width is 2, then after the first decision the algorithm keeps the two most likely partial trajectories of length one. Each of those is then extended by one more move, giving four partial trajectories of length two. The algorithm scores those four and keeps only the best two. At the next step, those two are again extended, producing four partial trajectories of length three, after which the best two are kept again. This process repeats until termination. Beam search with width 3 works in the same way, except that it keeps three partial trajectories at each stage, extends them all, and then prunes back to the best three. In this way, beam search explores a broader set of plausible futures than greedy decoding, which often improves solution quality, although it usually requires more computation per step.

Slide 12 evaluation metrics:

To compare these decoding strategies, I used four metrics. The first is completion rate: the proportion of planning queries for which the generated trajectory actually reaches the goal. The second is average steps to goal, computed over successful trajectories only. This measures path efficiency. I also measured average compute time per query and average compute time per step, because beam search is expected to be more computationally expensive than greedy decoding.

Slide 13 Dataset and evaluation approach:

The training data consisted of shortest valid trajectories generated for connected start-goal combinations across eight maze layouts. These trajectories were encoded into a discrete sequence format and used to train a small Transformer model. For this talk, you do not need the internal details of that model. What matters is that it processes a partial trajectory and produces probabilities for the next move.

For evaluation, I used five-fold cross-validation. In each fold, the model was trained on one partition of the trajectory dataset and tested on held-out start-goal queries from the same maze family. This allowed decoding strategies to be compared on planning problems that were not used in training for that fold, while still making use of the full dataset across the five evaluation rounds.

Slide 14: results:

The key result is that beam search outperformed greedy decoding. Aggregated across all eight maze layouts, greedy decoding achieved a completion rate of 0.6916, beam search with width 2 achieved 0.9100, and beam search with width 3 achieved 0.9754. Average steps to goal also improved, from 7.7205 for greedy decoding to 7.0771 for beam width 2 and 6.9058 for beam width 3. So the broader search not only reached the goal more often, but usually found shorter successful trajectories as well.

This trend was also consistent across the individual maze layouts. As expected, beam search required more computation per step. However, because it often reached the goal in fewer steps, its average total compute time per query was still competitive, and in my implementation it was even lower than greedy decoding in aggregate.

Slide 15: Future work:

The next stage of the project is to test how strongly these results depend on training on optimal trajectories. I also want to extend the work by studying additional decoding strategies, such as stochastic or nucleus-based decoding, and by exploring ways of combining model probabilities with reward-based guidance during decoding. Thank you for listening. I am happy to take any questions.